



BASICS

```
fn main() { println!("hi {}", 1 + 1); }

let x = 5;           // immutable
let mut y = 5;       // mutable
let z: u8 = 5;        // annotated
let x = x + 1;        // shadowing, may retype
const MAX: u32 = 100; // compile-time
static NAME: &str = "a"; // one address
```

SCALAR TYPES

i8 i16 i32 i64 i128 isize	signed
u8 u16 u32 u64 u128 usize	unsigned
f32 f64	float (default f64)
bool char	char is 4 bytes, Unicode
()	unit — "no value"

Default integer is `i32`. No implicit conversions: use `as` (lossy) or `try_into()` (checked). Overflow panics in debug, wraps in release — say what you mean with `checked_`, `saturating_`, `wrapping_`.

CONTROL FLOW

```
// if is an expression; condition must be bool
let s = if n > 5 { "big" } else { "small" };

loop { break value; } // breaks with a value
while cond { }
for i in 0..5 { } // 0..=5 includes 5
for x in &v { } // borrow
for x in &mut v { } // borrow mutably
for x in v { } // consume

'outer: for a in .. { break 'outer; }
```

FUNCTIONS

```
fn add(a: i32, b: i32) -> i32 {
    a + b // no semicolon = return value
}
fn early(x: i32) -> i32 {
    if x < 0 { return 0; } // leave early
    x
}
```

A semicolon discards a value. Missing `->` means the function returns `()`; a stray semicolon on the last line is why you get *expected i32, found ()*.

OWNERSHIP & BORROWING

- Every value has exactly one **owner**.
- Dropped when the owner leaves scope.
- Many `&T` or one `&mut T` — never both.
- A reference may not outlive its referent.

```
let s2 = s1; // MOVE: s1 unusable after
let s2 = s1.clone(); // deep copy, explicit cost
let r = &s1; // borrow: s1 still owner
let m = &mut s1; // exclusive borrow
```

TAKE	WHEN
<code>&T</code>	you only read — the default
<code>&mut T</code>	you modify in place
<code>T</code>	you store or consume it

Copy types (ints, floats, `bool`, `char`, `&T`, tuples of these) copy instead of moving. Borrows end at **last use**, not at the brace.

STRUCTS & ENUMS

```
#[derive(Debug, Clone, PartialEq)]
struct User { name: String, active: bool }
struct Meters(f64); // tuple / newtype
struct Marker; // unit struct

let u = User { name, active: true }; // shorthand
let v = User { active: false, ..u }; // update

impl User {
    fn new(name: &str) -> Self { .. } // assoc fn
    fn name(&self) -> &str { .. } // read
    fn rename(&mut self) { .. } // mutate
    fn into_name(self) -> String { .. } // consume
}

enum Message {
    Quit, // no data
    Move { x: i32, y: i32 }, // named
    Write(String), // tuple
}
```

PATTERN MATCHING

```
match value {
    1 => .., // literal
    2 | 3 => .., // alternatives
    4..=9 => .., // range
    n if n < 0 => .., // guard
    n @ 1..=5 => .., // bind + test
    Point { x, .. } => .., // destructure
    [a, .., z] => .., // slice
    _ => .., // catch-all
}

if let Some(n) = opt { .. } else { .. }
while let Some(x) = stack.pop() { .. }
let Some(n) = opt else { return; }; // diverges
matches!(v, Some(n) if n > 2) // -> bool
```

`match` is **exhaustive**: add an enum variant and every incomplete match becomes a compile error. That is the refactoring superpower — be sparing with `_`.

STRINGS

STRING	&STR
owned, heap, growable	borrowed view, fixed

```
s.to_string() / .to_owned() / String::from(s)
&s / s.as_str() // String -> &str
s.len() // BYTES, not chars
s.chars().count() // chars, O(n)
s.chars() / .bytes() / .char_indices()
s.split(',') / .split_whitespace() / .lines()
s.trim() / .replace(a,b) / .contains(p)
s.starts_with(p) / .find(p) / .split_once(p)
s.to_uppercase() / .parse::<i32>()
format!("{a} {b:?} {c:.2}")
```

No integer indexing — strings are UTF-8, so `s[0]` would be meaningless. Slice by byte range (`&s[0..5]`, panics off a char boundary) or use `.chars()`. Take `&str` in parameters.

COLLECTIONS

```
let mut v = vec![1, 2, 3];
v.push / pop / insert / remove / clear
v.len / is_empty / contains(&x)
v.sort() / sort_by(..) / sort_by_key(..)
v.dedup() / reverse() / retain(|x| ..)
v[0] // panics if absent
v.get(0) // -> Option<T>
&v[1..3] // slice, no copy

use std::collections::HashMap;
let mut m = HashMap::new();
m.insert(k, val); m.get(&k); m.remove(&k);
*m.entry(k).or_insert(0) += 1; // count
m.entry(k).or_default().push(v); // group
```

`HashMap` iteration order is deliberately random. Need order? `BTreeMap`. Membership only? `HashSet` / `BTreeSet`.

ERROR HANDLING

```
enum Option<T> { Some(T), None } // no null
enum Result<T, E> { Ok(T), Err(E) } // no throw

let n = text.parse::<i32>()?; // propagate
// ? == "unwrap, or return Err(e.into())"

.unwrap_or(d) / .unwrap_or_else(..)
.unwrap_or_default() / .ok()
.map(..) / .map_err(..) / .and_then(..)
.ok_or(e) / .ok_or_else(..) // Option -> Result
.expect("why I expected this") // panics
```

SITUATION	USE
Caller can handle it	<code>Result</code>
A bug / broken invariant	<code>panic!</code>
Library error type	<code>enum</code> + <code>thiserror</code>
Application error type	<code>anyhow</code>

Never `Err(String)` in a public API — callers end up matching on prose. `expect` over `unwrap`, always: the message is a comment the panic prints for you.

TRAITS & GENERICS

```
trait Shape {
    fn area(&self) -> f64; // required
    fn name(&self) -> String { .. } // default
}
impl Shape for Circle { .. }

fn f(x: &impl Shape) { } // static
fn f<T: Shape>(x: &T) { } // same thing
fn f<T>(x: &T) where T: Shape { } // same thing
fn f() -> impl Iterator<Item = u32> { }
let v: Vec<Box<dyn Shape>> = ..; // dynamic
```

DERIVE	GIVES YOU
<code>Debug</code>	<code>{:?}</code> — put it on everything
<code>Clone</code> , <code>Copy</code>	duplication
<code>PartialEq</code> , <code>Eq</code>	<code>==</code>
<code>PartialOrd</code> , <code>Ord</code>	<code><</code> , <code>sort()</code>
<code>Hash</code>	map/set keys
<code>Default</code>	<code>T::default()</code>

Implement `From`, never `Into` — you get `Into` free, plus the conversion `?` uses. **Orphan rule**: you must own the trait or the type; otherwise wrap it in a newtype.

CLOSURES & ITERATORS

```
let f = |x| x * factor; // captures
let f = move |x| x * factor; // takes ownership
// Fn: reads · FnMut: mutates · FnOnce: consumes

.map(..) .filter(..) .filter_map(..)
.flat_map(..) .flatten() .zip(..)
.enumerate() .rev() .chain(..) .peekable()
.take(n) .skip(n) .take_while(..) .step_by(n)
.fold(init, ..) .scan(..) .reduce(..)
.sum() .product() .count() .min() .max()
.any(..) .all(..) .find(..) .position(..)
.max_by_key(..) .partition(..) .unzip()
.collect::<Vec<_>>() // also String, HashMap,
// and Result<Vec<_>, E>
```

Adapters are **lazy** — nothing runs until a consumer does. Chains fuse into one loop, so they cost no more than the handwritten version. Delete any `collect()` in the middle of a chain.

SMART POINTERS

<code>Box<T></code>	heap, one owner; recursive types
<code>Rc<T></code>	many owners, 1 thread
<code>Arc<T></code>	many owners, many threads
<code>RefCell<T></code>	borrow check at runtime
<code>Mutex<T></code>	one writer, many threads
<code>RwLock<T></code>	many readers / one writer
<code>Cow<'a, T></code>	borrow, clone only if needed

`Rc<RefCell<T>>` = shared mutable, single thread.
`Arc<Mutex<T>>` = the same across threads. Cycles leak — break them with `Weak<T>`. Reaching for these everywhere usually means a `Vec` plus indices was the better design.

CONCURRENCY

```
let h = thread::spawn(move || { .. });
h.join().unwrap();

// scoped threads may borrow locals
thread::scope(|s| { s.spawn(|| ..); });

let (tx, rx) = mpsc::channel();
tx.send(v)?; for msg in rx { .. } // drop(tx)!

let c = Arc::new(Mutex::new(0));
*c.lock().unwrap() += 1; // unlocks on drop

async fn f() -> T { g().await }
tokio::join!(a(), b()) // truly concurrent
```

WORKLOAD	REACH FOR
CPU over a collection	<code>rayon</code> / <code>par_iter()</code>
A few long workers	threads + channels
Shared counter	atomics
Shared structure	<code>Arc<Mutex<T>></code>
1000s of IO waits	<code>async</code> + <code>tokio</code>

Send = movable between threads. **Sync** = shareable by reference. Both inferred; `Rc` and `RefCell` are neither, which is why misuse is a compile error. Never block inside `async`.

MODULES & CARGO

```
mod shapes; // loads shapes.rs
pub / pub(crate) / pub(super)
use crate::a::b as c;
pub use inner::Thing; // re-export
```

```
cargo check # fast type check
cargo run / build --release
cargo test # incl. doc tests
cargo fmt
cargo clippy -- -D warnings
cargo add serde -F derive
cargo doc --open
```

Everything is private by default — and `pub struct` does *not* make its fields public.

IDIOMS WORTH INTERNALISING

- Take `&str`, `&[T]`, `&Path` — never `&String`, `&Vec<T>`, `&PathBuf`.
- `as_` free · `to_` allocates · `into_` consumes · `try_` = fallible.
- Newtype over bare primitives; enums over boolean parameters.
- Make illegal states unrepresentable — then you cannot test for them because they cannot exist.
- Parse, don't validate: return a type that proves the check ran.
- `#[must_use]`, `#[non_exhaustive]`, and `Debug` on every public type.
- Read the compiler error. It names the binding and usually prints the fix.
- Read clippy. It is a free senior reviewer.
- Reaching for `c.clone()` to silence the borrow checker means the ownership design is unclear.
- Benchmarking a debug build is the most common performance mistake there is.